

Lab 1 Report:

Data Preparation Techniques for Machine Learning

Name:

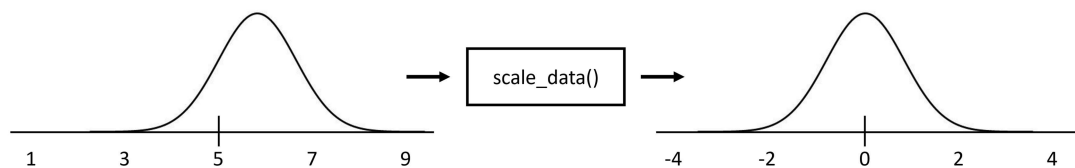
```
In [6]: # Import necessary libraries

%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
In [7]: from IPython.display import Image # For displaying images in colab jupyter cell
```

```
In [8]: Image('lab1_exercise1.PNG', width = 1000)
```

Out[8]:  Exercise 1: Scaling Data with Standard Scaling



- In Machine Learning, the dataset is usually scaled ahead of time so that it is easier for the computer to **learn** and **understand** the problem.
- One of the most frequently used method is 'standard scaling', where the data is scaled by $z = (x - \mu) / \sigma$. (x = original datapoint, μ = mean of the data, σ = standard deviation)
- Write a function "scale_data()" which takes 2D NumPy array as an input and perform standard scaling on its columns. The function should output a new 2D array containing scaled column data.
- Test your function with selected columns in CMS calorimeter dataset (hgcal.csv).
- Plot the scaled dataset for the selected columns by using the provided matplotlib histogram function.

66

```
In [58]: # Load the dataset (.csv) using pandas package

CMS_calori_dataset = pd.read_csv('hgcal.csv')

# .head directive on the panda dataframe displays the first n-rows

CMS_calori_dataset.head(n = 10)
```

Out[58]:

	Unnamed: 0	x	y	z	eta	phi	energy	t
0	0	179.50383	-23.632137	-7.878280	-0.0435	-0.130900	0.200126	4
1	1	-143.63881	110.217940	-72.706795	-0.3915	2.487094	2.734594	4
2	2	179.50383	-23.632120	-146.429610	-0.7395	-0.130900	0.423910	
3	3	-172.67310	54.443620	-238.065340	-1.0875	2.836160	0.713950	4
4	4	-180.88046	7.897389	-238.065340	-1.0875	3.097959	0.000000	4
5	5	-180.88045	-7.897438	-238.065340	-1.0875	-3.097959	0.034491	4
6	6	-152.69838	-97.279590	-265.020540	-1.1745	-2.574361	0.580138	4
7	7	-23.63213	179.503810	-325.172060	-1.3485	1.701696	0.411487	4
8	8	-152.69835	97.279594	89.977780	0.4785	2.574361	0.183141	
9	9	-176.76110	39.187016	107.930240	0.5655	2.923426	0.337551	

```
In [60]: # Convert the panda dataframe into numpy 2D array

CMS_calori_dataset_np = CMS_calori_dataset.to_numpy()

# The converted numpy array has the dimension of 420 (rows) x 8 (columns)

print(CMS_calori_dataset_np.shape)
```

(420, 8)

```
In [152... # Extract only x, y, z, eta, phi and energy columns from the dataset and stack t
# Name this new 2D array CMS_calori_dataset_np_sub.
# The array should have dimension 420 (rows) x 6 (columns)
arr_seq_1 = CMS_calori_dataset_np[:,1]
CMS_calori_dataset_np_sub = np.vstack([arr_seq_1, CMS_calori_dataset_np[:,2], CM
print(CMS_calori_dataset_np_sub.shape)
CMS_calori_dataset_np_sub = np.transpose(CMS_calori_dataset_np_sub)
print(CMS_calori_dataset_np_sub.shape)
print(CMS_calori_dataset_np_sub)
```

YOUR CODE HERE

(6, 420)

(420, 6)

```
[ [ 1.7950383e+02 -2.3632137e+01 -7.8782797e+00 -4.3500002e-02
   -1.3089974e-01  2.0012644e-01]
  [-1.4363881e+02  1.1021794e+02 -7.2706795e+01 -3.9149997e-01
    2.4870942e+00  2.7345939e+00]
  [ 1.7950383e+02 -2.3632120e+01 -1.4642961e+02 -7.3950000e-01
   -1.3089965e-01  4.2390990e-01]
  ...
  [-3.5694687e+01 -5.0977300e+01  5.2279800e+02  2.8250000e+00
   -2.1816616e+00  5.3157926e-01]
  [-1.6106770e+01 -6.0111294e+01  5.2279800e+02  2.8250000e+00
   -1.8325956e+00  1.4225300e-01]
  [ 6.0111294e+01 -1.6106758e+01  5.2279800e+02  2.8250000e+00
   -2.6179916e-01  2.0927284e+00]]
```

```
In [156... # Create the scaling function

def scale_data(arr):
    arr2d = np.zeros((len(arr),len(arr[0])))
    for i in range(0, len(arr[0])):
        column_means = np.mean(arr[:,i], axis=0)
        column_std = np.std(arr[:,i], axis=0)
        arr2d[:,i] = (arr[:,i] - column_means)/column_std
    return arr2d
```

```
In [158... # Test the function with CMS_calori_dataset_np_sub

CMS_calori_dataset_np_sub_scaled = scale_data(CMS_calori_dataset_np_sub)
```

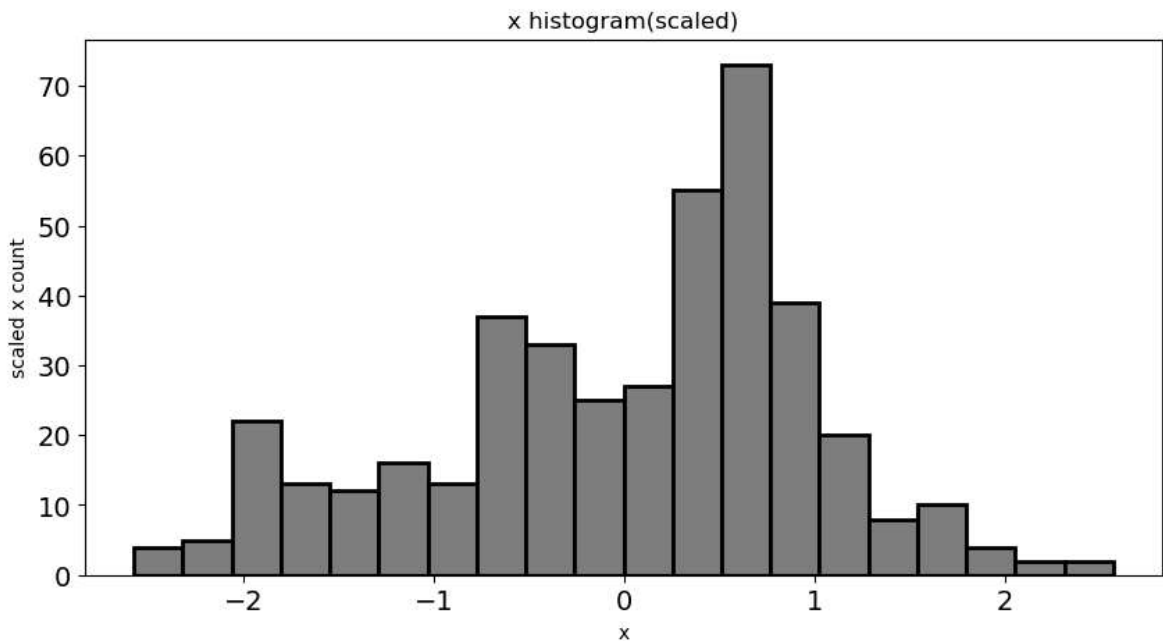
```
In [204... # Confirm the data is scaled for 'x' column

plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 0], bins = 20, facecolor = 'grey',
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add proper x-label and y-label
plt.xlabel("x")
plt.ylabel("scaled x count")
plt.title("x histogram(scaled)")
# YOUR CODE HERE

plt.show()
```



```
In [206... # Confirm the data is scaled for 'energy' column

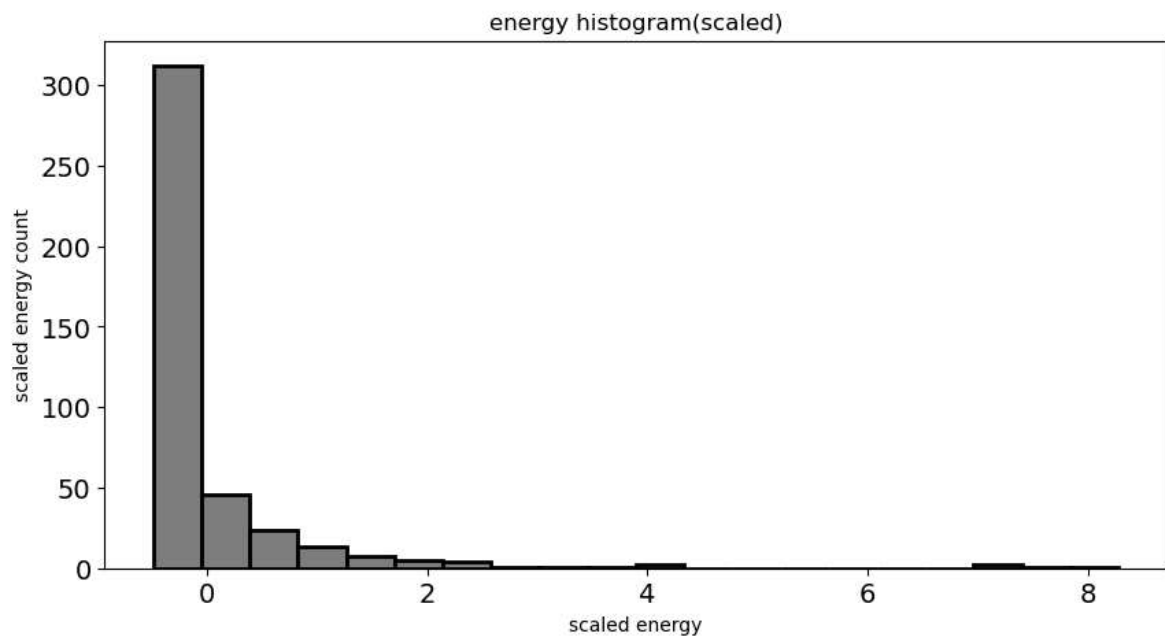
plt.figure(figsize = (10, 5))

plt.hist(CMS_calori_dataset_np_sub_scaled[:, 5], bins = 20, facecolor = 'grey',
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Add proper x-label and y-label
```

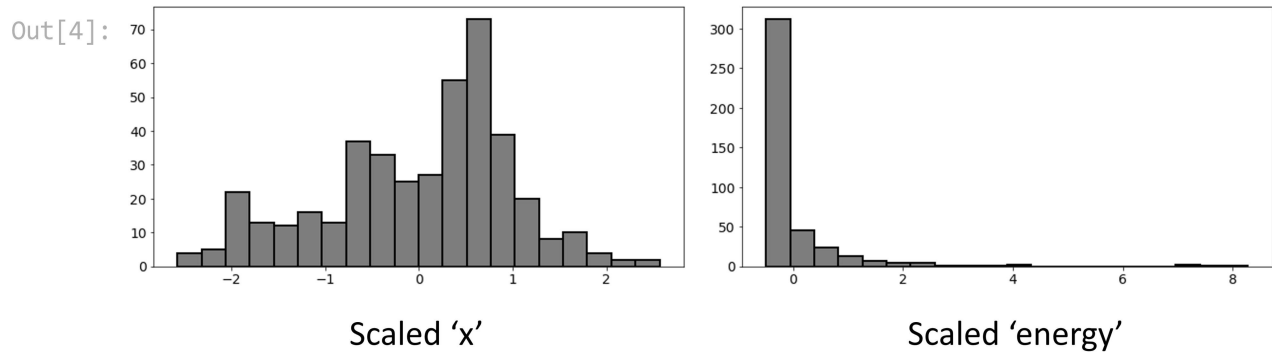
```
plt.xlabel("scaled energy")
plt.ylabel("scaled energy count")
plt.title("energy histogram(scaled)")
# YOUR CODE HERE

plt.show()
```



Expected histogram outputs - Feel free to style your plot differently

In [4]: `Image('lab1_e1_expected_outputs.PNG', width = 1000)`

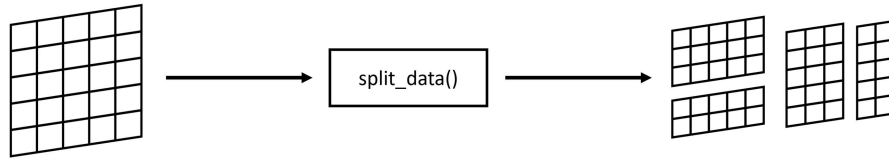


In [3]: `Image('lab1_exercise2.PNG', width = 1000)`

Out[3]:



Exercise 2: Data Splitting



- In this exercise you will write a function called `split_data()` which given a NumPy array, it splits the array into sub-arrays.
- Data splitting is used to divided the dataset into training, validation and testing sets, which we will describe in later lab.
- The function should take following parameters
 - `arr` – 2D NumPy array representing a dataset
 - `split_proportions` – a list containing split ratios, e.g., `[0.2, 0.3, 0.5]`
 - `axis` – a direction to be splitted (0 = row-wise, 1 = column-wise)
- Test your function on the scaled dataset from exercise 1 with given parameters in the lab template.
- Confirm that your sub arrays have correct dimensions by printing their shape

67

In [194...

Create the splitting function

```
def split_data(arr, split_proportions, axis):
    split_data_list = []
    if axis == 0:
        length = len(arr)
        pointer = 0
        ratio = 0
        for ele in split_proportions:
            ratio = ratio + ele
            upbound = int(length*ratio)
            split_data_list.append(arr[pointer:upbound,:])
            pointer = int(length*ratio)
    if axis == 1:
        length = len(arr[0])
        pointer = 0
        ratio = 0
        for ele in split_proportions:
            ratio = ratio + ele
            upbound = int(length*ratio)
            split_data_list.append(arr[:,pointer:upbound])
            pointer = int(length*ratio)

    # YOUR CODE HERE
    # Returns a List of numpy sub-arrays according to split proportions
    return split_data_list
```

In [196...

Test your split function against scaled CMS Calorimeter dataset from exercise

```
sub_data_list_1 = split_data(arr = CMS_calori_dataset_np_sub_scaled,
                             split_proportions = [0.6, 0.2, 0.2], axis =
```

In [198...

Confirm that dataset has been split into correct shapes

The correct dimensions should be (252, 6) (84, 6) (84, 6)

```
print(sub_data_list_1[0].shape, sub_data_list_1[1].shape, sub_data_list_1[2].sha

(252, 6) (84, 6) (84, 6)
```

```
In [200... # Test your split function against scaled CMS Calorimeter dataset from exercise  
  
sub_data_list_2 = split_data(arr = CMS_calori_dataset_np_sub_scaled,  
                             split_proportions = [0.5, 0.5],
```

```
In [202... # Confirm that dataset has been split into correct shapes  
# The correct dimensions should be (420, 3) (420, 3)
```

```
print(sub_data_list_2[0].shape, sub_data_list_2[1].shape)
```

```
(420, 3) (420, 3)
```